

在线学习平台

使用指南

涵盖学生、教师、管理员三种角色操作指引

一、学生操作指南

1. 如何进入学习

1. 在钉钉中打开「在线学习平台」H5 应用，自动登录
2. 在首页课程列表中选择一门课程
3. 点击课程进入详情页，查看所有小节
4. 点击小节进入学习页面，视频自动开始计时
5. 保持页面在前台且视频播放状态，系统每 30 秒记录一次有效学习时间

提示：暂停视频或切换到其他应用/标签页时，学习时间不会被计入。

2. 小节开播时间

部分小节设有「开播时间」，未到开播时间时无法进入学习，页面会显示锁定状态和倒计时。开播时间过后可随时进入（包括复习）。

3. 查看我的进度

点击底部导航「我的进度」可查看各课程的有效学习时长、学习百分比和完成状态。

4. 提交作业

1. 在课程详情页点击「作业」按钮进入作业页面
2. 每个小节可能有一份作业，点击进入可查看题目要求
3. 拍照或选择图片上传，点击提交
4. 截至日期前可多次提交，系统以最新版本为准

提示：迟交：截止时间后仍可提交，但会标记为「迟交」，教师可看到。

5. 签到日历

点击顶栏「签到」可查看本月学习签到日历，每天有有效学习记录即标记为已签到。

6. 学习报告

点击顶栏「报告」查看个人学习总结报告，包括总学习时长、各课程进度和推荐学习建议。

7. 小节评价

学完每节课后可以对小节进行评分（1-5 星）和文字评价，帮助教师改进课程。

二、教师操作指南

1. 创建和管理课程

1. 登录后在首页点击「创建课程」按钮
2. 填写课程标题、描述、要求学习时长等基本信息
3. 在课程编辑页添加小节，每个小节可设置独立视频
4. 视频支持两种方式：外部链接（B站/腾讯视频）或本地上传（MP4文件）
5. 可为每个小节设置「开播时间」，控制学生何时可以开始学习

2. 发布作业

1. 在课程详情页点击「作业管理」
2. 每个小节可创建一份作业，填写标题、题目描述和截止时间
3. 支持上传题目图片，设置评分标准
4. 发布后学生即可看到作业并提交

3. 查看学习统计

点击顶栏头像旁「学习统计」进入教师仪表盘，可查看：

- 各课程的学习人数、平均学习时长
- 学生完成率排名
- 班级维度统计

4. 发布公告

1. 点击顶栏「公告」进入公告列表
2. 点击「发布公告」撰写新公告
3. 可选择关联某门课程（仅该课程学生可见）或全平台公告
4. 设置优先级：普通/重要/紧急，紧急公告会醒目展示

5. 学习排行榜

在课程详情页点击「排行榜」查看该课程学生学习时长排名，按有效学习时长从高到低排列。

6. 学习报告

教师可查看班级维度的学习报告，了解全班学习进度、未完成学生名单和建议关注的学生。

7. 管理用户

在「管理后台」（教师/管理员可见）可查看所有用户列表、修改用户角色、分配班级、生成 API Key。

三、管理员操作指南

1. 管理后台

管理员拥有教师全部权限，额外可：

- 查看和管理所有用户（学生/教师）
- 修改任意用户角色
- 删除用户
- 分配和调整班级
- 生成/重置 API Key

2. API Key 使用

API Key 用于让智能体或其他程序调用平台接口，无需通过浏览器登录。

1. 在管理后台找到目标用户，点击「生成 API Key」
2. 复制生成的 Key（格式：sk_xxxx），只显示一次
3. 调用 API 时在请求头加入 X-API-Key: sk_xxxx
4. 所有需要认证的 API 都支持 JWT 和 API Key 两种认证方式

3. 全平台报告

管理员可查看全平台维度的学习报告，包括总用户数、活跃用户数、各课程完成率等全局数据。

4. 运维监控面板

点击顶栏「运维」进入监控面板，可查看：

- 系统健康状况（API 响应时间、数据库连接等）
- 在线用户实时统计
- 学习会话活跃度
- 服务器资源使用情况

5. 系统架构

技术栈	Python FastAPI + MySQL + Redis + Nginx
前端	Vue3 + Vite + DingTalk JSAPI（H5微应用）
部署	Docker Compose，服务器 115.223.38.90:1001
容器	study-monitor-backend / frontend / mysql / redis
数据库	MySQL 8.0，数据库名 study_monitor

6. 常用运维命令

```
# 查看容器状态
docker ps --filter name=study-monitor

# 重启后端容器（代码更新后）
cd /data/study-monitor && docker compose up -d --build backend frontend

# 查看后端日志
docker logs study-monitor-backend --tail 100 -f

# 进入 MySQL 容器
docker exec -it study-monitor-mysql mysql -u root -p

# 健康检查
curl http://127.0.0.1:8001/api/health
```

7. 代码更新部署流程

1. 本地提交代码并 push 到 GitHub
2. SSH 到服务器，cd /data/study-monitor
3. git pull（需要代理：https_proxy=http://127.0.0.1:7890）
4. 如有数据库变更，先执行迁移 SQL
5. docker compose up -d --build backend frontend
6. curl http://127.0.0.1:1001/ 验证首页可访问

8. 数据备份

```
# 导出数据库备份
```

```
docker exec study-monitor-mysql mysqldump -u root -p'Sm2026Root_Secure_Prod' study_monitor > backup_$(date +%Y%m%d).sql
```

恢复备份

```
docker exec -i study-monitor-mysql mysql -u root -p'Sm2026Root_Secure_Prod' study_monitor < backup_20260621.sql
```